Ошибки, организация кода, обобщения и коллекции

Продолжение курса
Zig 0.17.0
Теория и примеры для демонстрации

Команда 16
Гуцалов Егор
Крутянский Роман
Медведьков Роман

МАРШРУТ

Четыре блока лекции

01 Обработка ошибок и средства отладки
02 Пространства имен, модули и пакеты
03 Декларация и использование обобщений
04 Массивы, списки, карты и множества

Исключения и модель Zig

В Zig нет механизма throw / try-catch с раскруткой стека.

Ожидаемый сбой функция возвращает как значение.

Тип результата сообщает вызывающему коду о возможной ошибке.

Error set и error union

Error set перечисляет допустимые ошибки.

E!T содержит либо значение типа T, либо ошибку из E.

Ошибка сама по себе не хранит текст или объект с полями.

```zig
const ParseError = error{
    Empty,
    InvalidDigit,
};

fn digit(text: []const u8) ParseError!u8 {
    if (text.len == 0) return error.Empty;
    if (text.len != 1 or
        text[0] < '0' or text[0] > '9')
        return error.InvalidDigit;
    return text[0] - '0';
}
```

01_errors.zig

try передает ошибку вызывающей функции

При успехе try извлекает значение.

При ошибке немедленно выполняет return err.

Внешняя функция тоже должна уметь возвращать эту ошибку.

```zig
fn twice(text: []const u8) ParseError!u16 {
    const n = try digit(text);
    return @as(u16, n) * 2;
}

// Эквивалентная обработка:
const n = digit(text) catch |err| {
    return err;
};
```

01_errors.zig

catch восстанавливает выполнение

Обработчик может вернуть запасное значение.

Можно различать ошибки через switch.

Тип успешного результата и результат catch должны согласовываться.

```zig
const n = digit("") catch 0;

const value = digit("x") catch |err| blk: {
    break :blk switch (err) {
        error.Empty => @as(u8, 0),
        error.InvalidDigit => @as(u8, 9),
    };
};
```

01_errors.zig

Ветвление по error union

if разделяет успешный результат и ошибку.

!T позволяет компилятору вывести множество ошибок.

Для публичного API явное E!T точнее описывает контракт.

```zig
if (digit("7")) |n| {
    std.debug.print("digit={d}\n", .{n});
} else |err| {
    std.debug.print("error={s}\n",
        .{@errorName(err)});
}

fn readDigit(s: []const u8) !u8 {
    return digit(s);
}
```

01_errors.zig

defer и errdefer освобождают ресурсы

defer выполняется при обычном выходе из области, включая return с ошибкой.

errdefer выполняется при выходе с ошибкой.

Оба работают в обратном порядке регистрации.

```zig
fn copyChecked(a: std.mem.Allocator,
    text: []const u8) ![]u8
{
    const buf = try a.dupe(u8, text);
    errdefer a.free(buf);
    if (buf.len == 0) return error.Empty;
    return buf;
}

const owned = try copyChecked(a, "zig");
defer a.free(owned);
```

02_cleanup.zig

Ошибка, отсутствие значения и panic

E!T: операция может завершиться ожидаемым сбоем.

?T: значение может отсутствовать. Например, поиск ключа.

panic: нарушение инварианта или аварийное завершение.

catch работает с E!T, orelse работает с ?T.

assert фиксирует инвариант

std.debug.assert проверяет инвариант.

С runtime safety ложное условие вызывает panic.

Без safety ложное условие означает illegal behavior.

Пользовательский ввод проверяют через if и return error.

```zig
fn average(sum: u32, count: u32) u32 {
    std.debug.assert(count > 0);
    return sum / count;
}

test "average" {
    try std.testing.expectEqual(
        @as(u32, 5), average(10, 2));
}
```

03_assert.zig

Режим сборки меняет проверки

debug: проверки безопасности и удобная отладка.

safe: оптимизация с проверками безопасности.

fast и small: проверки по умолчанию отключены.

Названия приведены для Zig 0.17.0.

Диагностика и намеренный сбой

std.debug.print выводит значения в stderr.

@errorName помогает назвать ошибку.

@breakpoint останавливает процесс под отладчиком.

Error return trace показывает путь передачи ошибки.
Stack trace показывает цепочку активных вызовов.

```zig
std.debug.print("len={d}\n", .{items.len});

// Только отдельная демонстрация:
std.debug.assert(false);
```

negative/assert_fail.zig

Пространства имен

Контейнерные типы объединяют объявления.

Файл Zig тоже представляет контейнер с объявлениями.

Обращение через точку сохраняет происхождение имени.

```zig
const Units = struct {
    pub const scale: u32 = 1000;

    pub fn toBase(x: u32) u32 {
        return x * scale;
    }
};

const result = Units.toBase(3);
```

04_namespaces.zig

Файл как пространство имен

@import("math.zig") возвращает тип контейнера файла.

pub открывает объявление для других файлов.

Локальный псевдоним помогает сократить длинное имя.

```zig
// modules/math.zig
pub fn square(x: i32) i32 {
    return x * x;
}

// 05_import.zig
const math = @import("modules/math.zig");
const square = math.square;
const result = square(7);
```

05_import.zig

Namespace, module и package

Namespace: область имен объявлений контейнера.

Module: корневой файл, таблица именованных импортов и настройки компиляции.

Package: распространяемый проект с манифестом и зависимостями.

Пакет может предоставлять несколько модулей и артефактов.

Именованный импорт модуля

Сборка связывает имя mathlib с корневым файлом.

Исходный код импортирует устойчивое имя.

Путь к реализации остается в build.zig.

```zig
const lib = b.createModule(.{
    .root_source_file = b.path("modules/math.zig"),
    .target = target,
});
const root = b.createModule(.{
    .root_source_file = b.path("06_module.zig"),
    .target = target,
});
root.addImport("mathlib", lib);

// В 06_module.zig:
const math = @import("mathlib");
```

build.zig

Пакет и build.zig.zon

build.zig описывает шаги сборки и публикуемые модули.

build.zig.zon хранит метаданные и зависимости.

Локальная path-зависимость позволяет показать пакет без сети.

URL-зависимость фиксируют адресом и content hash.

```
// Фрагмент build.zig.zon:
.dependencies = .{
    .math_pkg = .{
        .path = "deps/math_pkg",
    },
},

// В build.zig:
const dep = b.dependency("math_pkg", .{});
root.addImport("pkgmath", dep.module("math"));
```

deps/math_pkg/build.zig

pub и export решают разные задачи

pub открывает имя для Zig-кода из другого файла.

export создает символ для линковщика.

extern объявляет реализацию, которую предоставит другая единица линковки.

Для внешнего ABI нужны совместимые типы.

```zig
// Доступ из другого Zig-файла:
pub fn square(x: i32) i32 {
    return x * x;
}

// Символ с C ABI:
export fn zig_add(a: i32, b: i32) i32 {
    return a + b;
}

// Внешняя реализация:
extern fn abs(x: c_int) c_int;
```

08_export.zig

Публичный интерфейс проекта

Корневой файл явно переэкспортирует нужные имена.

Внутренние детали остаются за границей интерфейса.

Пользователь импортирует один корень вместо дерева файлов.

```zig
// lib.zig
const impl = @import("internal/stack.zig");

pub const Stack = impl.Stack;
pub const version = "1.0.0";

// Потребитель:
const lib = @import("mylib");
const IntStack = lib.Stack(i32, 8);
```

Тип как параметр времени компиляции

comptime T: type принимает тип во время компиляции.

Функция может вернуть новый тип.

Типы и параметры специализации не хранятся в каждом экземпляре.

```zig
fn Box(comptime T: type) type {
    return struct {
        value: T,
    };
}

const IntBox = Box(i32);
const x = IntBox{ .value = 42 };
const y = Box(bool){ .value = true };
```

09_generics.zig

Инстанцирование и создание значения

Box(i32) задает конкретную специализацию типа.

Box(i32){ .value = 42 } создает экземпляр.

Box(bool) задает другую структуру данных.

Параметром может быть и число, известное при компиляции.

```
fn Buffer(comptime T: type,
    comptime capacity: usize) type
{
    return struct {
        data: [capacity]T,
    };
}

const Small = Buffer(u8, 16);
const Large = Buffer(u8, 64);
```

09_generics.zig

Обобщенная функция

Явный параметр T делает тип частью контракта.

anytype выводит тип аргумента.

Операции проверяются при анализе конкретной специализации.

```zig
fn max(comptime T: type, a: T, b: T) T {
    return if (a > b) a else b;
}

fn identity(value: anytype) @TypeOf(value) {
    return value;
}

const n = max(i32, 7, 12);
const ok = identity(true);
```

09_generics.zig

Ограничения через comptime

@typeInfo позволяет проверить категорию типа.

@compileError формирует понятное сообщение.

Универсальность стоит ограничивать требованиями алгоритма.

```zig
fn requireInteger(comptime T: type) void {
    switch (@typeInfo(T)) {
        .int => {},
        else => @compileError(
            "T must be an integer type"),
    }
}

comptime { requireInteger(u32); }
// requireInteger(f32) даст ошибку компиляции.
```

negative/generic_fail.zig

Пример: стек фиксированной емкости

T определяет тип элементов.

capacity определяет размер встроенного массива.

@This() ссылается на возвращаемый тип.

len отделяет заполненную часть от свободной.

```zig
pub fn Stack(comptime T: type,
    comptime capacity: usize) type
{
    return struct {
        const Self = @This();
        data: [capacity]T = undefined,
        len: usize = 0,

        // Методы на следующем слайде.
    };
}
```

10_stack.zig

Методы стека и явные ошибки

push возвращает Full при заполнении.

pop возвращает Empty при отсутствии элементов.

Успех меняет len и сохраняет инвариант.

```zig
pub fn push(self: *Self, value: T)
    error{Full}!void
{
    if (self.len == capacity) return error.Full;
    self.data[self.len] = value;
    self.len += 1;
}

pub fn pop(self: *Self) error{Empty}!T {
    if (self.len == 0) return error.Empty;
    self.len -= 1;
    return self.data[self.len];
}
```

10_stack.zig

Использование и цена специализации

Один исходный код работает с разными типами.

Проверки типов происходят при компиляции.

Возможны дополнительные затраты времени компиляции и рост машинного кода.

Конкретный объем кода зависит от оптимизатора.

```
var ints: Stack(i32, 2) = .{};
try ints.push(10);
try ints.push(20);
const top = try ints.pop(); // 20

var flags: Stack(bool, 4) = .{};
try flags.push(true);
```

10_stack.zig

Карта выбора контейнера

[N]T: фиксированное число элементов внутри значения.

[]T: представление существующей последовательности.

ArrayList(T): растущая последовательность.

HashMap(K, V): поиск по ключу.

HashMap(K, void), EnumSet, BitSet: множества.

Массив хранит элементы в значении

Размер входит в тип: [3]i32 и [4]i32 различаются.

[_] позволяет вывести длину из инициализатора.

Массив не изменяет размер после создания.

```zig
var a = [_]i32{ 10, 20, 30 };
a[1] = 25;

var total: i32 = 0;
for (a) |value| total += value;

std.debug.print("len={d}, sum={d}\n",
    .{ a.len, total }); // 3, 65
```

11_arrays.zig

Срез представляет часть массива

[]T хранит указатель и длину.

Границы диапазона: начало включено, конец исключен.

[]const T запрещает изменение элементов через этот срез.

Изменение mutable-среза меняет исходный массив.

```zig
var a = [_]i32{ 10, 20, 30 };
const view: []i32 = a[1..3];
view[0] = 99;
// a == { 10, 99, 30 }

const read_only: []const i32 = view;
std.debug.print("first={d}\n",
    .{read_only[0]});
```

11_arrays.zig

ArrayList увеличивает емкость

items содержит текущие элементы.

capacity описывает размер доступного хранилища.

append может выделить память и вернуть OutOfMemory.

В используемом API allocator передают в операции.

```zig
const a = std.testing.allocator;
var list: std.ArrayList(i32) = .empty;
defer list.deinit(a);

try list.append(a, 10);
try list.append(a, 20);
try list.appendSlice(a, &.{ 30, 40 });

try std.testing.expectEqual(
    @as(usize, 4), list.items.len);
```

12_arraylist.zig

Владение и действительность указателей

Рост ArrayList может сделать старые указатели недействительными.

ensureTotalCapacity резервирует хранилище.

Контейнер не освобождает вложенные ресурсы элементов автоматически.

Для них нужна отдельная очистка.

```zig
try list.ensureTotalCapacity(a, 16);
try list.append(a, 10);

// После получения указателя:
const ptr = &list.items[0];
_ = ptr;
// Рост, перемещение или удаление элемента
// требуют заново получить нужный указатель.
```

12_arraylist.zig

Карта с числовыми ключами

AutoHashMap автоматически выбирает хеширование и сравнение подходящих ключей.

put добавляет или заменяет значение.

get возвращает ?V, contains проверяет наличие.

Порядок обхода не задает сортировку.

```zig
var ages = std.AutoHashMap(u32, u8)
    .init(std.testing.allocator);
defer ages.deinit();

try ages.put(101, 20);
try ages.put(101, 21);

try std.testing.expectEqual(
    @as(u8, 21), ages.get(101).?);
try std.testing.expect(ages.get(999) == null);
```

13_maps.zig

Строковые ключи и время жизни

StringHashMap сравнивает содержимое строк.

Обычная StringHashMap хранит срез ключа без автоматического копирования байтов.

Литералы живут достаточно долго. Для временных буферов нужна явная стратегия владения.

```zig
var scores = std.StringHashMap(u32)
    .init(std.testing.allocator);
defer scores.deinit();

try scores.put("zig", 1);
const entry = try scores.getOrPut("zig");
if (!entry.found_existing)
    entry.value_ptr.* = 0;
entry.value_ptr.* += 1;
```

14_stringmap.zig

Множество через HashMap(K, void)

Значение void не несет пользовательских данных.

Ключи определяют элементы множества.

Повторная вставка не увеличивает число элементов.

contains и remove задают основные операции.

```zig
var set = std.AutoHashMap(u32, void)
    .init(std.testing.allocator);
defer set.deinit();

try set.put(7, {});
try set.put(7, {});
try set.put(9, {});
try std.testing.expectEqual(
    @as(u32, 2), set.count());
_ = set.remove(7);
```

15_sets.zig

EnumSet и битовые множества

EnumSet(E) подходит для набора значений enum.

StaticBitSet(N) представляет индексы ограниченного диапазона.

Битовые операции позволяют быстро объединять наборы.

HashMap удобнее для разреженных произвольных ключей.

```zig
const Color = enum { red, green, blue };
var colors = std.EnumSet(Color).initEmpty();
colors.insert(.red);
colors.insert(.blue);
try std.testing.expect(colors.contains(.red));

var bits = std.StaticBitSet(64).initEmpty();
bits.set(5);
try std.testing.expect(bits.isSet(5));
```

15_sets.zig

Другие структуры стандартной библиотеки

PriorityQueue: извлечение элемента по приоритету.

Deque: вставка и удаление с обоих концов.

MultiArrayList: отдельное хранение полей структур.

StaticStringMap: таблица строковых ключей, заданная при компиляции.

Выбор зависит от операций, владения и размера данных.

Сложность основных операций

Массив / срез: доступ по индексу O(1).

ArrayList: append амортизированно O(1), рост O(n).

ArrayList: удаление с сохранением порядка O(n).

HashMap: поиск в среднем O(1), худший случай O(n).

Битовый набор: проверка одного индекса O(1).

Общий пример: частоты слов

StringHashMap связывает слово со счетчиком.

try передает ошибки выделения памяти.

defer освобождает хранилище карты.

Входные литералы обеспечивают время жизни ключей.

```zig
var counts = std.StringHashMap(u32).init(a);
defer counts.deinit();

const words = [_][]const u8{
    "zig", "code", "zig",
};
for (words) |word| {
    const e = try counts.getOrPut(word);
    if (!e.found_existing) e.value_ptr.* = 0;
    e.value_ptr.* += 1;
}
// counts.get("zig").? == 2
```

16_wordcount.zig

Официальная документация

Zig 0.17.0 Language Reference
ziglang.org/documentation/0.17.0/

Standard Library 0.17.0
ziglang.org/documentation/0.17.0/std/

Zig Build System
ziglang.org/learn/build-system/

Дата подготовки: 18 октября 2026